Artificial Intelligence

BS (CS) _SP_2026

Lab_01 Manual

Learning Objectives:

1. Python Basic
2. Operators
3. Variables
4. Data Type
5. Control Statements
6. List, Sets, Tuples, Dictionaries
7. Classes and Objects
8. Local and Global Variables
9. Oop Concepts (Polymorphism, Abstraction, Inheritance, Encapsulation)

Lab Manual
Basis of Python

Variables
In Python, variables are used to store and manage data. Each variable has a name and a value, and the type of the variable indicates what kind of data it can hold.

Variable Naming Rules:

- Variable names in Python can contain alphanumerical characters a-z, A-Z, 0-9 and some special characters such as _.
- Normal variable names must start with a letter.
- By convention, variable names start with a lower-case letter, and Class names start with a capital letter.
- In addition, there are a number of Python keywords that cannot be used as variable names. These keywords are:

and, as, assert, break, class, continue, def, del, elif, else, except, exec, finally, for, from, global, if, import, in, is, lambda, not, or, pass, print, raise, return, try, while, with, yield

Syntax:

```
a = 10

name = "John"

is_student = True
```

```
#Taking input from a user

name = input ("Enter student name: ")
```

Operators
Python language supports the following types of operators.

- Arithmetic Operators
- Comparison (Relational) Operators
- Assignment Operators
- Logical Operators
- Membership Operators

Arithmetic Operators:

Operator	Example
Addition	a + b = 30
Subtraction	a - b =10
Multiplication	a * b
Division	a / b
Modulus	b % a
Exponent	a ** b

Comparison Operators:

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true	a == b
!=	If values of two operands are not equal, then condition becomes true.	a != b
< >	If values of two operands are not equal, then condition becomes true.	a < > b
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	a > b
<	If the value of left operand is less than the value of right operand, then condition becomes true.	a < b
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	a >= b
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	a <= b

Logical Operators:

Operator	Description	Example
and	If both the operands are true, then condition becomes true.	a and b
or	If any of the two operands are non-zero, then condition becomes true.	a or b
not	Used to reverse the logical state of its operand	a not b

Membership Operators:

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	if a in b then: statement
not in	Evaluates to true if it does not find a variable in the specified sequence and false otherwise.	if a not in b then: statements;

Control Statements

In Python, control statements are used to guide the program's flow of operations. Depending on specific conditions, they choose which statements to perform first or repeatedly go through a list of statements.

If-else Statement:

```
if condition:
    # Statements
elif another_condition:
    # Statements
else:
   #Statemets
```

For Statement

```
item_list = []
for variable in item_list:
        #Statements
```

While Statement

```
while condition:
    # Code to be executed as long as the condition is true
else:
```

List

List are ordered collection of data. They are mutable (changeable) and allow duplicate elements. They are created using square brackets []. Data can be different types. Lists are versatile and can be used to store and manipulate sequences of items.

Syntax

```
list_A = ['a','b','c','d','BS_CS', 0343]
print(list_A)
print(list_A[1:4])      or      print(list_A[:4])      #Slicing
```

Different operations can be performed on list

Operation	Description	Syntax
Append	Add an element at the end	list_A.append(1,2,3)
Insert	Inserts an element at a specific index	list_A.insert(1, 'Python')

Extend	Adds more than one element at the end of the list	list_A.extend([‘Python’, ‘Java’ , ‘Perl’])
Remove	Removes the first item with the specified value	list_A.remove(‘Perl’)
Pop	Removes last element or removes the element at the specified position.	list_A.pop(); list_A.pop(2); #Remove?
Index	Returns the index of the first element with the specified value	list_A.index(0343);
Sorting	Sorts the list. And reverse() reverses the order of the list	list_A.sort(); list_A.reverse();

Built-in Functions:

Function	Description
cmp (list1, list2)	Compares elements of both lists.
len (list1)	Gives the total length of the list.
max (list1)	Returns item from the list with max value.
min (list1)	Returns item from the list with min value.
list (seq)	Converts a tuple into list.

Set

Set is unordered collection of data. They are mutable and don’t allow duplicate elements (Unique elements). They are created using curly braces {} or the set() constructor. Sets are useful for mathematical operations like union, intersection, and difference.

basket = {‘Apple’, ‘Orange’, ’pearl’, ‘Banana’, 1, 2, 57}

Different operations can be performed on set

Operation	Description	Syntax
Union	All the elements of Set1 or Set2	new_set = Set1 | Set2 Set1.union(Set2)
Intersection	Include the common elements between the two sets.	new_set = Set1 **&** Set2 Set1.intersection(Set2)
Difference	Elements of set1 that are not present on set2.	new_set = Set1 - Set2 Set1.difference(Set2) #Output {1,3,4,5,6} Set2.difference(Set1) #Output?
Symmetric difference	all elements of set1 and set2 without the common elements	new_set = Set1 ^ Set2 Set1.symmetric_difference(Set2)
Subset	Returns True if another set contains this set	new_set = Set1 <= Set2 Set1.issubset(Set2)
Superset	Returns the index of the first element with the specified value	new_set = Set1 >= Set2 Set1.issuperset(Set2)
Add	Sorts the list. And reverse() reverses the order of the list	new_set.add(4)
Remove	Removes the element from a set. It returns a Key Error if element is not part of the set.	new_set.remove(2)
Discard	Removes the element from the set, and doesn't raise the error	new_set.discard(3)
Clear	Removes all elements from the set	new_set.clear()

Tuples

Tuples are ordered, immutable (unchangeable), and allow duplicate elements. They are created using parentheses (). Tuples are often used for fixed collections of items where immutability is desired.

data = ('Apple', 'Orange', 'pearl', 'Banana', 1, 2, 57)

```
new_tuple = data [1:4]          #create a new tuple by extracting a portion of an existing tuple using slicing.
```

Data Manipulation

Data manipulation is used for modifying or transforming data to get relevant information, conduct analyses, or prepare data for future processing. There are various Python libraries that are used for data manipulation, widely used library is Pandas for data manipulation and analysis.

Install libraries

```
pip install numpy               #IDLE
!pip install numpy              #Google collab
```

Creating Array

```
import numpy as np
data1 = np.array ([1, 2, 3, 4, 5])                  #1D array
data2 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  #2D array
 data3 = np.array([1, 2, 3], [4, 5, 6], [7, 8, 9])   #1D or 2D?
```

Indexing

In python indexing starts from 0

```
data1[0];
data2[0][1];
data3[0][0];
```

Slicing

Data Frames

```
import pandas as pd
# Creating a DataFrame from a dictionary
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'Salary': [50000, 60000, 75000]}
df = pd.DataFrame(data)
```

Classes

A class is a blueprint for creating objects. It serves as a template that defines the attributes and behaviors of an object.

It encapsulates data (attributes) and methods (functions) that operate on that data.

Also, promote code reusability and maintainability.

Syntax:

```
class ClassName:
        #class attributes and methods
        def functionName:
```

Objects

Objects are instances of a class, created using the class name followed by parentheses.

Objects encapsulate the attributes and methods defined in the class.

Syntax:

```
object_name = ClassName()
my_car = Car("Toyota", "Camry", 2022)
```

Local and Global Variables

Outside the class

```
# Global variables for Employee class
variable1 = "value1"
variable2 = value2


class ClassName:
    def __init__(self, variable1, variable2):
        self.variable1 = variable1
        self.variable2 = variable2
    def display():
```

```
    print(f" Variable 1: {variable1},\nVariable 2: {variable2}”)
```

Inside the class

```
class ClassName:
variable1 = "value1"
variable2 = value2
  def __init__(self, variable1, variable2):
    self.variable1 = variable1
    self.variable2 = variable2
  def display():
    print(f" Variable 1: {variable1},\nVariable 2: {variable2}”)
```

Inside the function

```
def some_function():
  local_variable = 12
  # local_variable is a local variable
```

OOP Concepts

Python is a multi-paradigm programming language. Meaning, it supports different programming approach. One of the most popular approach to solve a programming problem is by creating objects. This is known as Object-Oriented Programming (OOP).

An object has two characteristics:

• Attributes

• Actions (behavior)

The concept of OOP in Python focuses on creating reusable code.

This concept is also known as **DRY (Don't Repeat Yourself**).

Object Oriented programming is a programming style that is associated with the concept of Class, Objects and various other concepts revolving around these two, like Inheritance, Polymorphism, Abstraction, Encapsulation etc.

Classes

A class is a blueprint for creating objects. It serves as a template that defines the attributes and behaviors of an object.

It encapsulates data (attributes) and methods (functions) that operate on that data. Also, promote code re-usability and maintainability.

Syntax:

```
class ClassName:
#class attributes and methods
def functionName:
```

Example:

```
class Dog:
    def __init_(self, name):
        self.name = name
    def bark(self):
        return f"{self.name} says woof!"
dog1 = Dog("Buddy")
dog2 = Dog("Max")
print(dog1.bark())
print(dog2.bark())
```

Objects:

An object is an instance of a class. It is a concrete entity created based on the blueprint defined by the class.

Inheritance:

Inheritance is a mechanism where a new class (subclass) can inherit properties and behaviors from an existing class (superclass). This promotes code reusability and allows for the creation of specialized classes.

Syntax:

```
class SubClassName(SuperClassName):
# Subclass definition
# Additional methods/attributes here
Pass
```

Example:

```
class Animal:
    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def bark(self):
        return "Dog barks"

dog = Dog()
print(dog.speak())
print(dog.bark())
```

Encapsulation:

Encapsulation refers to the bundling of data (attributes) and methods that operate on the data within a single unit (class). It hides the internal state of an object from the outside world and allows controlled access to it through methods.